G

实验：系统项目

注意：这些项目正在陆续添加到 `https://github.com/remzi-arpacidusseau/ostep-projects`，那里有项目说明和一个简单的测试框架。如果你感兴趣，一定要去看看。

本章给出一些系统项目的思路。在一个 15 周的学期里，我们通常做六七个项目，也就是大约每两周一个。前几个项目通常由学生独自完成，最后几个则两人一组完成。

每个学期，项目都沿用这个大纲；不过我们会改变一些细节，让项目保持新鲜感，也让跨学期“共享”代码更难一些（当然，没有人会干这种事！）。我们还会用 Moss 工具 [M94] 来查找这类“共享”。

至于评分，我们试过许多不同的方法，各有利弊。现场演示很有意思，但很花时间。自动化测试脚本没那么耗时，但要让它们仔细测试那些有意思的边界情况，得花很大的心思。关于这些项目的更多细节，请查看本书的网页；如果你想要自动化测试脚本，我们很乐意分享。

G.1 入门项目

第一个项目是系统编程入门。典型的作业是在不同的约束条件下，写一个 `sort` 工具的某种变体。例如，对文本数据排序、对二进制数据排序，以及其他类似的项目都很合适。要完成这个项目，你需要熟悉一些系统调用（以及它们返回的错误码），用到几个简单的数据结构，除此之外没有太多别的。

G.2 UNIX Shell

在这个项目中，学生要实现一个 UNIX shell 的变体。学生会学到进程管理，还会弄明白管道和重定向这些神秘的东西实际上是怎么工作的。变体包括一些不寻常的功能，比如一个同时用 gzip 压缩输出的重定向符号。另一个变体是批处理模式，允许用户把几个请求攒成一批再执行，执行时也许还会采用不同的调度策略。

G.3 内存分配库

这个项目通过构建一个替代的内存分配库（类似 `malloc()` 和 `free()`，只是名字不同），探索一块内存是怎样管理的。学生会学到怎样用 `mmap()` 获取一块匿名内存，然后为了构建一个简单的（也可能更复杂的）空闲链表来管理这块空间，还会非常深入地学习指针。变体包括：最佳适应 / 最坏适应、伙伴分配，以及其他各种分配器。

G.4 并发入门

这个项目介绍用 POSIX 线程进行并发编程。构建一些简单的线程安全的库：链表、散列表，以及一些更复杂的数据结构，都是给实际代码加锁的好练习。测量粗粒度和细粒度两种做法的性能。变体只是换成不同的（也许更复杂的）数据结构。

G.5 并发 Web 服务器

这个项目探索并发在实际应用中的用法。学生拿一个简单的 Web 服务器（或者自己写一个），给它加上一个线程池，以便并发地处理请求。线程池的大小应该固定，并用一个生产者/消费者有界缓冲区把请求从主线程传给固定数量的工作线程。学生会学到怎样用线程、锁和条件变量来构建一个真正的服务器。变体包括线程的各种调度策略。

G.6 文件系统检查程序

这个项目探索磁盘上的数据结构及其一致性。学生要实现一个简单的文件系统检查程序。在 Linux 上可以用 `debugfs` 工具制作真实的文件系统镜像；遍历这些镜像，确保一切正常。要想更难一些，还可以修复发现的问题。变体侧重于不同类型的问题：指针、链接计数、间接块的使用等等。

G.7 文件系统碎片整理程序

这个项目探索磁盘上的数据结构及其对性能的影响。项目应该给学生一些已知有碎片问题的特定文件系统镜像；然后学生遍历镜像，找出没有顺序存放的文件。写出一个解决了这个问题的新的“碎片整理后”的镜像，也许还可以报告一些统计数据。

G.8 并发文件服务器

这个项目把并发和文件系统结合起来，甚至还涉及一点网络和分布式系统。学生要实现一个简单的并发文件服务器。协议应该和 NFS 差不多，支持查找、读、写和获取文件属性。把文件存放在一个磁盘镜像里（这个镜像本身就是一个文件）。变体有很多，可以建议不同的磁盘格式和网络协议。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[M94] “Moss: A System for Detecting Software Plagiarism” Alex Aiken. Available: http://theory.stanford.edu/~aiken/moss/

 © 2008–23, ARPACI-DUSSEAU